# Extracted Test Questions

Cleaned and organized from the uploaded 16-page scan. Answer hints from scenario pages were omitted so this PDF contains questions/prompts only.

## 1. Core Java

### Fundamentals & OOP

- Difference between Abstract Class and Interface.
- Can we achieve multiple inheritance in Java? How? (Interfaces)
- Explain OOPs concepts with real-time examples.
- What is the difference between Encapsulation vs Abstraction?
- Why is String immutable? What are the benefits?
- Difference between == and equals()?
- What is a transient variable?
- Can we override a static method?
- What is the purpose of hashCode() and equals()? How do they work internally?
- Difference between Comparable and Comparator.
- What is a marker interface? Give examples.

### Collections & Internal Working

- Difference between HashMap and ConcurrentHashMap.
- How does HashMap work internally? Explain buckets, hashing, and resizing.
- Explain the differences between HashMap, LinkedHashMap, and TreeMap.
- Fail-fast vs Fail-safe iterators (with real examples).
- ArrayList vs LinkedList - when to use each.
- HashSet vs TreeSet.
- How does Set work internally? What happens if you modify an object after adding it to a HashSet?
- ConcurrentModificationException - when does it occur?

### Java 8 & Functional Programming

- What are functional interfaces? Why are they needed for lambdas?
- Difference between Predicate, Function, Supplier, Consumer.
- What is the Function<T, R> interface?
- Java Stream API - purpose and examples.
- Difference between map() and flatMap().
- How to implement custom collectors in Java streams.
- Optional - how to avoid NullPointerException effectively.
- Stream coding tasks: count character occurrences, find first non-repeated character, find duplicates, max/min, group by, partition, etc.
- Find the second highest number using streams.
- Group anagrams using streams.

- First occurrence mapping (first word starting with each letter).
- Sum and average of all elements.
- Reverse an integer array.
- Palindrome check using streams.
- Fibonacci series using Stream.iterate.
- Remove duplicates, join strings with delimiter, flatten list of lists.

### Java Versions & New Features

- Java 8 -> 11 -> 17 -> 21 -> 25 evolution (key features).
- What are records in Java and when would you use them over a traditional class?
- What are sealed classes? How do they differ from abstract or final classes?
- What are virtual threads in Java 21? How do they help in high-concurrency systems?
- Pattern matching in Java 16+ - how does it simplify control flow?
- Java 11 features: var, new HTTP Client, etc.
- Java 17 features: sealed classes, pattern matching for instanceof, text blocks.
- Java 21 features: virtual threads, pattern matching for switch, record patterns.
- Java 25 focus: Project Panama, Project Valhalla.

### Memory Management & JVM

- How does Garbage Collection work in Java? Types and algorithms.
- Explain JVM memory structure (Heap, Stack, Metaspace).
- What causes OutOfMemoryError in production? How to debug?
- How to decide JVM memory allocation? Considerations while allocating memory.
- What is a memory leak? How to identify and fix?
- How does the Java compiler work? Interpreter vs JIT.
- ClassLoaders - purpose and types.

### Multithreading & Concurrency

- What is a race condition? How to prevent it?
- How to implement multithreading in Java? Thread vs Runnable vs Callable.
- Difference between sleep() and wait().
- What is a deadlock? How to prevent it?
- What is a mutex? What is a semaphore?
- What is the volatile keyword? What does it guarantee?
- What are Atomic classes? When are they used?
- synchronized vs volatile vs AtomicInteger.
- synchronized block vs method.
- How ConcurrentHashMap achieves thread safety internally.
- ThreadLocal - benefits and pitfalls.
- ExecutorService - types and use cases. How to size thread pools for CPU-bound vs I/O-bound?
- CompletableFuture - asynchronous programming, error handling, timeouts.

- CountDownLatch - use case.
- Producer-Consumer problem implementation.
- How to make a class thread-safe? Give a real scenario.
- How to handle shared mutable state in high-traffic microservices.
- Debugging race conditions that appear only in production.
- Java Memory Model - visibility, happens-before relationship.

### Exception Handling

- Checked vs Unchecked exceptions.
- Output and execution order of try-catch-finally.
- How to handle exceptions in a multithreaded environment?

### Design Patterns & Principles

- SOLID principles - explain with real-life examples from your project.
- When to use creational, structural, behavioral patterns.
- Singleton pattern (thread-safe implementation).
- Factory pattern - how do you know which subclass to create?
- Builder pattern - use case.
- Adapter pattern - real example.
- Observer, Strategy, Command, State, etc.
- Design patterns used in microservices (Circuit Breaker, Saga, etc.).

## 2. Spring & Spring Boot

### Core Concepts & Annotations

- What is Spring Boot? Why is it preferred over Spring Framework?
- Explain Spring Boot Auto Configuration - how does it work internally?
- What are Spring Boot Starters?
- Difference between @Component, @Service, @Repository, @Controller, @RestController.
- What is the @SpringBootApplication annotation composed of?
- How does Dependency Injection work internally in Spring?
- What is the IoC container? Bean lifecycle - init and destroy.
- Bean scopes (singleton, prototype, request, session, etc.).
- How to create/get a Spring Bean? Different ways.
- @Autowired - internal working.
- @Qualifier vs @Primary - when to use each.
- @Bean vs @Component.
- @Value - injecting values from properties.
- @PropertySource, @ConfigurationProperties.
- @Profile - environment-specific configuration.
- @Conditional annotations (@ConditionalOnProperty, @ConditionalOnClass).

- @PostConstruct - purpose and usage.

### Web Layer & REST APIs

- @RequestMapping, @GetMapping, @PostMapping, etc.
- Difference between PUT and PATCH.
- @PathVariable, @RequestParam, @RequestBody.
- @ResponseBody - how it works.
- How to version REST APIs (URI, parameter, headers).
- Pagination and sorting in REST APIs.
- How does @RequestBody work internally?
- HTTP status codes (4xx, 5xx) - real-world usage.
- Idempotency - which HTTP methods are idempotent and why.

### Exception Handling & Validation

- Global exception handling with @ControllerAdvice and @ExceptionHandler.
- Custom error responses.
- @Valid and validation annotations (@NotNull, @Size, etc.).

### Security

- How to secure REST APIs in Spring Boot? (Spring Security, OAuth2, JWT)
- Explain JWT authentication flow step-by-step.
- How does Spring Security validate a JWT token?
- What is SecurityContext and SecurityContextHolder?
- Spring Security filter chain.
- Session-based vs token-based authentication.
- Method-level security vs role-based security.
- OAuth2 - authentication vs authorization.
- Certificate-based authentication.
- SSO login flow.

### Transaction Management

- What is @Transactional? How does it work internally?
- Propagation levels and isolation levels.
- Rollback rules - when does rollback happen?
- @Transactional on private/protected methods - does it work?
- Nested calls and transaction boundaries.

### Caching & Scheduling

- @EnableCaching, @Cacheable, @CachePut, @CacheEvict.
- @EnableScheduling, @Scheduled (cron, fixed delay).
- @EnableAsync, @Async - asynchronous processing.
- Scheduling impact on API latency - how to isolate.

### Actuator & Monitoring

- What is Spring Boot Actuator? Name some endpoints.
- Custom actuator endpoints.
- Custom health indicators.
- How to expose metrics with Prometheus and Grafana.

### Production & Debugging

- How to debug a Spring Boot application that works locally but fails in production?
- How to identify bottlenecks when CPU is low but requests timeout?
- application.properties changes not reflected - why?
- Multiple beans of same type cause startup failure - how to fix?
- Bean initialization issues - how to detect?
- What happens if @PostConstruct throws an exception?
- Duplicate bean registration in multi-module projects - cause and fix.
- How to manage feature toggles safely?
- How to safely reload configs without restarting?
- Classloader issues in fat JARs.

### Testing

- Unit testing with JUnit and Mockito.
- Difference between @Mock and @InjectMocks.
- Integration tests with @SpringBootTest, @WebMvcTest, @DataJpaTest.
- @MockBean - injecting mocks into Spring context.
- @TestConfiguration for custom test configuration.

## 3. Microservices & Cloud

### Architecture & Design

- Monolithic vs Microservices architecture - advantages and disadvantages.
- How to decompose a monolith into microservices? Strangler pattern.
- How do microservices communicate with each other? Synchronous vs asynchronous.
- REST vs gRPC - when to use which?
- What is API Gateway? Why is it used? Examples (Zuul, Spring Cloud Gateway).
- Service discovery - why needed? Eureka, Consul.
- Configuration management - Spring Cloud Config Server.
- Centralized logging and distributed tracing (ELK, Jaeger, Zipkin).
- Monitoring - Prometheus, Grafana, Micrometer.

### Resilience & Fault Tolerance

- Circuit Breaker pattern - states (Closed, Open, Half Open). Implementation with Resilience4j/Hystrix.
- Retry mechanism with delay.
- Fallback methods - graceful degradation.

- Bulkhead pattern.
- How to handle partial failures in microservices.
- What is the Saga pattern? Orchestration vs Choreography.
- Distributed transactions - 2-Phase Commit vs Saga.
- Idempotency - how to ensure safe retries (e.g., payment creation).
- Handling duplicate messages in Kafka.

### Communication Patterns

- Synchronous: RestTemplate, Feign Client, WebClient.
- Asynchronous: Message queues (Kafka, RabbitMQ), WebFlux.
- Event-driven architecture - when to choose messaging over REST.
- Kafka - delivery guarantees (at-most-once, at-least-once, exactly-once).
- Kafka consumer lag - how to diagnose and fix.
- Kafka vs Redis Pub/Sub.

### Deployment & Cloud

- Deploying Spring Boot microservices on AWS (EC2, ECS, Lambda).
- Containerization with Docker - Dockerfile, image storage.
- Kubernetes basics - pods, services, ConfigMap, Secret, Ingress.
- CI/CD pipeline - Jenkins, GitHub Actions.
- How to roll back a failed deployment.
- Blue-green deployment, canary releases.

### Real-World Scenarios

- If multiple microservices are failing, how do you identify the root cause?
- How to ensure zero data loss?
- How to handle concurrent updates to the same database row across services?
- How to scale a microservice to handle millions of requests per minute?
- Designing a global rate limiter across distributed instances.

## 4. Databases & Persistence

### SQL & NoSQL

- SQL vs NoSQL - when to use which.
- Database optimization - indexing, query tuning, avoiding unnecessary joins.
- Difference between INNER JOIN, LEFT JOIN, RIGHT JOIN.
- WHERE vs HAVING.
- Find 2nd highest salary - SQL query.
- Find employees earning more than department average.
- Self join - use cases.
- Stored procedures - how to create and use.
- Primary key vs unique key vs index.

- Clustered index vs non-clustered index.
- Drop, truncate, delete - differences.
- How to manage schema changes without breaking production.

### JPA & Hibernate

- JPA entity lifecycle (Transient, Persistent, Detached, Removed).
- Lazy vs eager loading - performance implications.
- N+1 query problem - cause and fix.
- @Transactional and rollback.
- First-level cache (session) vs second-level cache.
- save(), persist(), saveAndFlush() - differences.
- How does an Entity class eventually become a table? (JPA -> Hibernate -> SQL)
- Configuration required to connect to a database.

### Data Consistency & Transactions

- How to handle concurrent updates to the same database row (optimistic/pessimistic locking).
- ACID properties with real-world examples.
- Distributed transactions across microservices - challenges and patterns (Saga, 2PC).
- How to ensure data consistency without distributed transactions.

## 5. DevOps, CI/CD & Tools

### Topics

- CI/CD pipeline - tools and stages.
- Git - branching strategies, merge conflicts, stash.
- Maven vs Gradle - build tools.
- Docker - containerize a Spring Boot application.
- Kubernetes - basic commands, pod lifecycle.
- Jenkins - pipeline scripting.
- SonarQube, Checkstyle - code quality.
- ELK/EFK - centralized logging.
- Prometheus & Grafana - monitoring and alerting.

## 6. Data Structures, Algorithms & Problem Solving

### DSA Core Topics

- Arrays: sliding window, prefix sum, Kadane's algorithm.
- Linked Lists: reverse, cycle detection (Floyd's), merge point.
- Stacks & Queues: min stack, deque, priority queue.
- Trees: traversals, diameter, LCA, balanced tree check.
- Graphs: BFS, DFS, topological sort, cycle detection, Dijkstra.
- Dynamic Programming: LCS, LIS, 0/1 knapsack, matrix chain.
- Hashing: HashMap, HashSet, collision handling.

- Tries: insert, search, prefix matching.
- Recursion & Backtracking: N-Queens, permutations, subsets.

### Coding Problems (from interviews)

- Find all duplicate elements in an array.
- Remove duplicates from sorted array (allow at most two duplicates).
- Thread-safe singleton implementation.
- Best Time to Buy and Sell Stock.
- Merge two sorted linked lists.
- 2Sum, 3Sum - combinations that sum to target.
- Find first non-repeating character in a string.
- Group anagrams.
- Find maximum employee salary per department (Streams or SQL).
- Minimum platforms / interval scheduling.
- Sliding window maximum.
- Copy linked list with random pointer.
- 132 pattern detection.
- Rotated sorted array search.
- Longest time window maintaining SLA.
- Pyramid pattern problem.

## 7. System Design

### Fundamentals

- CAP theorem - trade-offs.
- Load balancing (L4, L7) - reverse proxy (Nginx).
- Caching strategies (Redis, CDN, Memcached).
- Database sharding, partitioning, replication.
- Consistent hashing.
- Quorum reads/writes, gossip protocol.
- Message queues - Kafka, RabbitMQ.
- Event-driven architecture, event sourcing, CQRS.
- Distributed locks (Zookeeper, Redis).
- Consensus algorithms (Paxos, Raft).

### Design Problems

- Design a rate-limiting solution for high-traffic API.
- Design a transaction-processing backend with consistency and retries.
- Design a write-intensive server (trade-offs).
- Design BookMyShow system (HLD).
- Design a near real-time data pipeline (Kafka -> stream processing -> time-series DB -> Grafana).

- Design a global rate limiter across distributed instances.
- Design authorization models (RBAC / ABAC).

### Production & Observability

- How to enforce API SLAs at runtime (Prometheus + Grafana).
- How to identify and fix a production latency spike.
- How to design monitoring, alerting, and dashboards.
- How to implement centralized logging for faster incident diagnosis.
- Circuit breakers and retries - protecting downstream services.

## 8. Behavioral & Project Experience

### Questions

- Self introduction.
- Explain your current project - architecture, tech stack, role.
- What are your roles and responsibilities?
- What challenges have you faced in your project? How did you solve them?
- How do you handle team conflicts or misunderstandings between teams?
- How do you mentor junior developers?
- Explain a real production incident you solved.
- How do you ensure security in your APIs?
- What performance optimizations have you done?
- Have you used AI tools to improve productivity? How?
- Strengths and weaknesses.
- Questions for the interviewer.

## 9. Debugging & Troubleshooting Tools

### Production Issue Diagnosis

- Logging & Log Aggregation: ELK Stack, EFK, Splunk, and correlating logs across microservices.
- Application Performance Monitoring (APM): Prometheus + Grafana, Micrometer, Dynatrace, New Relic, Datadog.
- Jaeger, Zipkin - distributed tracing (OpenTelemetry).
- Profiling & Memory Analysis: JProfiler, VisualVM, Eclipse MAT, jmap, jstat, jstack.
- Heap dump vs thread dump - when to use which.
- How to capture a heap dump in production without restart.
- Analyzing OutOfMemoryError - heap dumps, GC logs.
- GC Log Analysis: enabling GC logs, GCViewer, GCEasy, interpreting GC pauses/throughput/heap sizing.
- Thread Dump Analysis: jstack or kill -3, identifying deadlocks/thread contention/blocked threads, FastThread/TDA.
- Network & API Debugging: Wireshark, tcpdump, Postman, curl, Fiddler, Charles Proxy.
- Database Query Analysis: slow query logs, EXPLAIN/EXPLAIN ANALYZE, pgAdmin, MySQL Workbench, New Relic DB.
- JVM Internal Inspection: jcmd, jinfo, jstat, jconsole.
- Container & Cloud Debugging: Docker logs/exec/stats, kubectl logs/describe/exec, AWS CloudWatch/X-Ray.

- Profiling in Production: Async Profiler and Flight Recorder (JFR).

### Common Debugging Scenarios

- Memory leak - how to detect (heap dump analysis, GC logs).
- CPU spike - thread dump analysis to find hot threads.
- Latency spike - combined analysis of logs, metrics, traces.
- API timeout - tracing across services, checking thread pool utilization.

## 10. Real-World Production Scenarios

### API Performance & Slow Endpoints

- Your API was fast in testing, but in production it starts timing out after a few hours. What would you check?
- CPU usage is low, but API latency is high. How do you investigate?
- A specific endpoint becomes slower over time until a restart fixes it. What is happening?
- You deploy a new version, and suddenly response times increase by 30%. How do you roll back or fix without causing more downtime?
- Your API returns correct data but response time fluctuates wildly. How do you pinpoint the cause?

### Application Startup & Lifecycle

- Your Spring Boot application takes 10 minutes to start in production but starts quickly locally. Why?
- The application starts successfully but after a few days, new requests are not being processed. How do you debug?
- You notice that the application fails to start because of a missing configuration property, but the property is present in the config server. Why might this happen?

### Fixing Issues Without Downtime

- You find a critical bug in production that causes wrong data to be saved. How do you fix it without taking the system down?
- A third-party API you depend on starts failing intermittently. How do you prevent your service from going down?
- You need to change a database schema without any downtime. How do you handle that?
- A memory leak is detected in production - heap keeps growing until the pod is killed. How do you diagnose and fix without restarting every few hours?
- You have to apply a security patch to a running application without restarting. Is it possible?

### Observability & Proactive Detection

- How do you get alerted before a production issue becomes a full outage?
- Your logs are missing in production but present locally. What could be the cause?
- You see intermittent 500 errors, but they do not correlate with any deployment. How do you find the root cause?

### Handling Partial Failures & Data Consistency

- Two microservices communicate via HTTP. The downstream service fails mid-transaction. How do you ensure data consistency?
- Kafka consumers are lagging, and messages are being processed slowly. How do you increase throughput without losing messages?
- A payment transaction is initiated but the user's balance is deducted, yet the confirmation fails. How do you recover?
- You push a configuration change to the config server, but microservices do not pick it up. How do you force a reload without restart?